

2.4 Exemplul 2 - Conflicte

Se considera gramatica al carei limbaj e 0^{2n} .

$S \rightarrow 0 S 0$

$S \rightarrow \epsilon$

Regulile corespunzatoare yacc sunt:

```
9  S: '0' S '0'
10 | /* epsilon */
11 ;
```

Figura 2: Descrierea Yacc pentru gramatica 0^{2n}

Odata invocat yacc, se obtine:

```
yacc -d -v first.y
```

```
first.y: warning: 1 shift/reduce conflict [-Wconflicts-sr]
```

Prima linie din fisierul y.output descrie locatia si cauza conflictului, in cazul nostru:

State 1 conflicts: 1 shift/reduce

Fisierul generat listeaza regulile gramaticale, plus o extra regula generata de yacc. Se listeaza terminalii si non-terminalii, dupa care se listeaza starile parser-ului alaturi de expresiile punct asociate lor.

Pentru starea 1, in cadrul careia exista un conflict, prima linie contine descrierea starii prin notatie punct. Mai departe, conform productiei $S \rightarrow '0' S '0'$, daca urmeaza un '0' la intrare, se poate realiza o actiune shift, iar automatul ar tranzitiona in starea 1. In acelasi timp, conform productiei $S \rightarrow \epsilon$, daca urmeaza un '0' la intrare, se poate realiza o reducere. Aceasta situatie marcheaza un **conflict**. Ultima linie corespunzatoare starii 1, "S go to state 3" se refera la o tranzitie spre starea 3, in urma actiunii de reducere.

Fisierul y.output arata astfel:

```
State 1 conflicts: 1 shift/reduce

Grammar
  0 $accept: S $end
  1 S: '0' S '0'
  2 | %empty
  ...
State 1
  1 S: '0' . S '0'
  '0' shift, and go to state 1
  '0' [reduce using rule 2 (S)]
  S go to state 3
  ...
```

Listing 2: Fisierul y.output pentru gramatica 0^{2n}

2.4.1 Conflicte Shift/Reduce

Apare cand intr-o stare se poate realiza atat actiune de shift cat si de reduce. Pentru exemplul prezentat anterior, starea 1 e o stare in care s-a inceput recunoasterea unui S cu regula 1, s-a consumat un 0 si se asteapta recunoasterea unui S (cel de dupa .) urmat de un 0. Se poate observa ca Yacc considera doua posibilitati pentru aceeasi intrare '0':

1. Caracterul '0' care urmeaza in input indica ca se va recunoaste un nou S format conform primei reguli ($S \rightarrow 0S0$); acest nou S e cel mentionat dupa punctul descrierii starii 1 ($S \rightarrow '0' \cdot S '0'$). In acest caz, parser-ul realizeaza shift pentru tokenul de intrare si incepe recunoasterea unui S de forma 0S0.
2. Caracterul '0' in input e vazut ca ultimul '0' din $S \rightarrow '0' \cdot S '0'$, pentru cazul in care S din partea dreapta a productiei s-ar descompune prin productia $S \rightarrow \epsilon$. In acest caz, parser-ul ar realiza o actiune reduce.

2.4.2 Conflicte Reduce/Reduce

Acest tip de conflict poate aparea atunci cand cel putin doua productii gramaticale pot fi folosite pentru o actiune de reducere intr-o anumita stare a automatului. De exemplu, considerand gramatica de mai jos:

$S \rightarrow T$
 $S \rightarrow \epsilon$
 $T \rightarrow 0 \ 0 \ T$
 $T \rightarrow \epsilon$

Descrierea acestei gramatici in Yacc ar face ca la compilare sa apara un conflict de tip reduce/reduce deoarece la intalnirea in intrare a unui token EOF, acel token ar putea fi derivat fie din S, fie din T, rezultand astfel doua posibilitati de actiuni reduce.

2.5 Eliminarea Conflictelor

Analizand automatul generat, putem sa gasim motivul pentru conflictele ce pot aparea, si se pot face eventuale modificari la gramatica sau la fisierul yacc pentru eliminarea conflictului si pastrarea limbajului. Ca si tehnici, amintim urmatoarele:

1. **Restructurarea Gramaticii:** Se poate schimba gramatica pentru a elimina sursa conflictului. In exemplul din Figura 2, motivul conflictului shift/reduce este acela ca in starea $S \rightarrow '0' \cdot S '0'$ nu se poate determina daca o intrare '0' vine de la recunoasterea lui S (shift), sau de la finalul productiei curente, in cazul in care s-ar recunoaste ϵ ca S (reduce).

Astfel, s-ar putea restructura gramatica in felul urmator, pentru a elimina conflictele:

$S \rightarrow '0' '0' S$
 $S \rightarrow \epsilon$

2. **Informatii Aditionale catre Yacc:** Se pot oferi informatii aditionale Yacc-ului pentru ca acesta sa decida care dintre actiunile in conflict ar trebui aleasa in situatii ambigue.

In general, ambiguitatea apare atunci cand nu exista destule informatii despre **precedenta** si **asociativitatea** unor operatori folositi in expresii. Aceste tipuri de conflicte shift/reduce se



Figura 3: Variante de parsare ce arata ambiguitatea operatorilor

pot elimina folosind declaratii ca `%left`, `%right` sau `%prec`.

De exemplu, sa consideram gramatica de mai jos:

$E \rightarrow E \text{ '}' + \text{'}$ E

$S \rightarrow \text{id}$

Atunci cand parser-ul intalneste `'+'`, va exista un conflict intre posibilitatea ca `'+'`-ul sa fie recunoscut ca parte din $E \rightarrow E \text{ '}' + \text{'}$ E (shift), conform Figurii 3 a, dar si posibilitatea ca el sa fie recunoscut dintr-o ramura recursiva a lui E, precum in Figura 3 b (reduce).

Acest conflict se poate rezolva si rescriind gramatica, dar se poate solutiona mai usor indicand asociativitatea dorita pentru operatorul `'+'`.

3. **Default:** Yacc-ul va alege o actiune implicita

- Intr-un conflict shift-reduce se alege shift.
- Intr-un conflict reduce-reduce se alege regula gramaticala care apare prima (in secventa de intrare).

E important de retinut ca nu toate conflictele pot fi eliminate. Pentru unele gramatici, pastrarea echivalentei in urma eliminarii conflictelor nu este posibila.

~~3 Desfasurarea Laboratorului~~

~~3.1 Functionare yacc~~

~~Analizati modul de functionare al analizorului sintactic folosind y.output pentru fisierele first si pentru exemplul DING-DONG.~~

~~3.2 Expresii Matematice~~

~~Analizati fisierele dc1.l si dc1.y. analizati rezultatele pe diverse expresii: $2+3*4$ si $2*3+4$. Se obtine rezultatul corect? Propuneti solutii de corectare~~

~~3.3 Database Configuration~~

~~Analizati utilizarea union din exemplul db.config din laboratorul din saptamana 4.~~